

TRACE: Targeted Ranking-Aware Counterfactual Explanation for Sequential Recommendation

Ungsik Kim

Gyeongsang National University
Jinju-si, Republic of Korea
blpeng@gnu.ac.kr

Gun-Woo Kim

Gyeongsang National University
Jinju-si, Republic of Korea
gunwoo.kim@gnu.ac.kr

Sang-Min Choi

Gyeongsang National University
Jinju-si, Republic of Korea
jerassi@gnu.ac.kr

Suwon Lee

Gyeongsang National University
Jinju-si, Republic of Korea
leesuwon@gnu.ac.kr

Abstract

Ranking-constraint counterfactual explanation for sequential recommendation requires query-limited search to decide where to edit and what to substitute—the bottlenecked for query efficiency lies more in how the search space is structured than in the mutation rate alone. We propose **TRACE** (Targeted Ranking-Aware Counterfactual Explanation), which decomposes the search into three stages under an embedding-accessible, training-free setting: influence-guided position selection, plausibility-aware candidate retrieval, and actual-margin beam pruning. Across five datasets and three recommender architectures, TRACE outperforms the GA-based baseline GECE in validity, query efficiency, edit cost, and likelihood preservation, with the largest gains on bring-in (raising a target item to top-1) and up to an order-of-magnitude reduction in queries on push-out (displacing the current top-1 beyond top-K). Ablations confirm that the gains arise from structuring the search space before evaluation, rather than from increased mutation frequency. Available code: <https://github.com/devUuung/TRACE>

CCS Concepts

• Information systems → Recommender systems.

Keywords

sequential recommendation, counterfactual explanation, ranking constraint, discrete search

ACM Reference Format:

Ungsik Kim, Sang-Min Choi, Gun-Woo Kim, and Suwon Lee. 2026. TRACE: Targeted Ranking-Aware Counterfactual Explanation for Sequential Recommendation. In *20th ACM Conference on Recommender Systems (RecSys '26)*, September 27–October 02, 2026, Minneapolis, MN, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3773078.3831796>

Corresponding author: leesuwon@gnu.ac.kr.



This work is licensed under a Creative Commons Attribution 4.0 International License. *RecSys '26, Minneapolis, MN, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2284-4/2026/09

<https://doi.org/10.1145/3773078.3831796>

1 Introduction

Sequential Recommender Systems (SRS)—building on a lineage from collaborative filtering [14, 24] to neural sequence models such as GRU4Rec [7], SASRec [9], and BERT4Rec [27]—achieve high next-item prediction accuracy by directly exploiting interaction ordering [23]. However, these models are opaque: their internal decision-making is difficult to interpret, making it difficult to answer “why was this item recommended?” This black-box nature constrains user trust, operator error analysis, and regulatory compliance [1, 33], and hinders the alignment between users’ mental models and the actual recommender.

Users and operators of recommender systems often care not only about whether a recommendation changes, but about how the ranking changes in specific directions. For instance, a service operator launching a new product may want to understand which user interactions would be necessary to elevate that item into the top-1 position; conversely, a user dissatisfied with dominant recommendations in a particular category may wish to identify which past actions keep that item at the top.

Users also hold individual mental models of how the service works—deliberately clicking the like button under the assumption that the service recommends based on such interactions, or intentionally engaging with content opposite to their usual preferences to steer recommendations [3, 17]. When the actual recommender is misaligned with these mental models, trust and satisfaction diminish; when alignment is achieved, users can understand the causes behind the recommender’s predictions and actively participate in shaping the recommendations they receive [15, 16].

Counterfactual explanation (CE) is a promising approach to address this problem [2, 10, 21, 30]. CE presents alternative scenarios obtained by minimally modifying the original user sequence to change the recommendation outcome, thereby providing causal intuition that links behavioral changes to recommendation changes. For example, a user may recognize that “I only watched action movies recently, so romance is not being recommended,” thereby identifying which actions drive the current recommendation.

Nevertheless, CE research for SRS remains in its early stages: neither the search methods nor the evaluation criteria have been sufficiently established [2, 6, 26, 28]. First, existing methods lack a search strategy that systematically organizes which positions to modify first and which items to consider as candidates. Second,

evaluation has primarily focused on recommendation-change success (Validity/Fidelity) and edit distance (Hamming/edit distance), leaving underexplored criteria that capture the naturalness of generated sequences [20, 33]. Third, because the discrete candidate space must be searched directly, query costs grow rapidly, making efficiency improvements critical in query-limited settings. This work primarily addresses the first and third limitations through structured search, and partially addresses the second by introducing a likelihood-based proxy for sequence naturalness.

In response, this work explicitly formalizes two ranking-constraint counterfactuals: bring-in, which raises a specified item to top-1, and push-out, which pushes the current top-1 item outside the top-K. These two settings require stronger ranking constraints than the untargeted flip formulation, which only checks whether the top-1 has changed. In a query-limited environment, the critical bottleneck is therefore not simply increasing mutation frequency, but rather deciding which positions to open and which candidates to send for actual evaluation. Starting from this diagnosis, we propose TRACE, a structured searcher that organizes the search into staged decisions before performing actual evaluation. TRACE adopts an embedding-accessible, training-free setting, requiring only score queries and item embedding similarity without access to model weights—a regime that covers realistic deployments where item embeddings are served separately or the underlying model is open-source, while remaining strictly less demanding than full white-box access.

- **Diagnosis.** We identify that the primary bottleneck in query-limited ranking-constraint CE is organizing the search space before evaluation—not increasing mutation frequency—and formalize this as a three-stage decomposition: position opening, candidate proposal, and actual-margin pruning.
- **Method.** We propose **TRACE**, which instantiates this decomposition as influence-guided position selection, plausibility-aware candidate retrieval via embedding similarity, and actual-margin beam pruning under an embedding-accessible, training-free setting that requires no access to model weights.
- **Validation.** Experiments across five datasets and three recommender architectures show that TRACE consistently outperforms the GA-based baseline GECE in validity, query efficiency, edit cost, and likelihood preservation, with the largest validity gains on bring-in and the largest efficiency gains on push-out.

2 Related Work

2.1 Explainability in Sequential Recommendation

As sequential recommenders such as GRU4Rec [7], SASRec [9], and BERT4Rec [27] have achieved stronger predictive performance, explaining their decisions has become a central research concern [33]. A common approach interprets attention weights as indicators of which items influenced a recommendation; however, Jain and Wallace [8] showed NLP tasks that attention weights are frequently uncorrelated with gradient-based feature importance and that different attention distributions can yield identical predictions, raising doubts about attention-based explanations more broadly, including in neural recommenders. Feature attribution methods such as

SHAP [18] and LIME [25] offer model-agnostic alternatives, but they are commonly applied under feature-independence assumptions that do not capture the temporal dependencies inherent in interaction sequences. These limitations motivate counterfactual explanations, which ask directly what minimal input changes would alter the recommendation. In the ranking-constraint SRS setting, the harder question is how to find such changes efficiently under a strict query budget.

2.2 Counterfactual Explanation for RS

A counterfactual explanation (CE) provides explanatory power by presenting an alternative scenario obtained through minimal modification of the input to change the model’s output [4, 10, 30]. In the recommendation domain, methods such as PRINCE [2] and CountER [28] propose modifying non-sequential item features or weights to induce recommendation changes. ACCENT [29] extends this to neural recommenders by applying influence-function-style scoring to guide item removal, but remains in the untargeted setting without ranking constraints. In the NLP domain, methods such as Polyjuice [31] frame discrete counterfactual generation as controlled token substitution—an analogous formulation to item-sequence editing in SRS. In the sequential recommendation setting, however, counterfactual generation is considerably more challenging due to the temporal dependencies in sequences and the large discrete item space [6, 26]. The recently proposed GECE [26] applies a genetic algorithm (GA) to search for counterfactual sequences without accessing the model’s internals; however, because it repeatedly applies undirected mutation and crossover, its query efficiency and validity degrade sharply in settings that require specific ranking constraints. More fundamentally, existing SRS CE methods do not explicitly structure the search space to reduce query cost: GA-based approaches rely on undirected operations, while learning-based approaches such as CETD [6] embed plausibility implicitly in a trained generator rather than as an explicit retrieval filter. In the information retrieval domain, CFE2 [32] addresses a related ranking-flip objective by editing the query; TRACE tackles the analogous problem on the interaction sequence, which introduces different temporal and combinatorial structure than query editing.

2.3 Algorithmic Recourse and Plausibility

Beyond validity and sparsity, the broader algorithmic recourse literature has established that counterfactual explanations should also satisfy feasibility and actionability constraints [10, 30]. FACE [22] formalizes this by requiring counterfactuals to follow data-manifold paths, preventing moves to unreachable points in the input space, while Karimi et al. [11] distinguish counterfactual explanations from actionable recourse by incorporating causal structure. In the recommendation domain, analogous requirements take the form of plausibility with respect to the user’s interaction history and the item distribution [20, 34]: a counterfactual sequence should remain behaviorally coherent rather than an arbitrary combination of items the user would be unlikely to interact with. Yet no existing SRS CE method operationalizes this principle as a pre-evaluation filter on the candidate pool, relying instead on undirected mutation over the full item space.

2.4 Metrics for Counterfactual Explanations

The standard CE evaluation metrics—validity, sparsity, and feasibility [10, 21, 30]—have been adopted in sequential-recommendation CE studies as validity, Hamming distance, and fidelity@ k [2, 6, 26, 28]. However, existing evaluations have largely focused on final success and edit size [12], and do not reveal which stage of query-limited search is the actual bottleneck. Recent benchmarking and plausibility discussions have argued for jointly considering semantic consistency, likelihood-based plausibility, and actionability [20, 34]. Following this line, we additionally report ΔNLL as a model-based likelihood proxy for sequence naturalness, complementing validity and edit distance. Because TRACE makes the stages of search explicit, we further conduct a stage-wise ablation that isolates the contributions of position selection, candidate proposal, and pruning — an analysis that has not been feasible under the monolithic search strategies of prior work.

3 Preliminary

Let $\mathcal{U}$ be a set of users and $\mathcal{I}$ a set of items. We denote by $\mathcal{H}_u \subseteq \mathcal{I}$ the item set of user u 's full interaction history. The model input sequence $S = [s_1, \dots, s_l] \in \mathcal{I}$ is defined as the most recent l items from the user's full history, where s_j is the j -th element of the truncated sequence actually fed to the model. An SRS model $\mathcal{M}$ takes S as input and produces a score vector $\mathbf{z}(S) \in \mathbb{R}^{|\mathcal{I}|}$ over all items. We define the recommendable item set as $\mathcal{O}_S := \mathcal{I} \setminus \mathcal{H}_u$, and denote by $\mathcal{O}_S = [o_1^S, o_2^S, \dots]$ the ranking obtained by sorting $\mathcal{O}_S$ in descending order of score, so that $o_1^S \in \mathcal{O}_S$ is the next-item prediction. The distance function $\delta : \tilde{\mathcal{I}}^l \times \tilde{\mathcal{I}}^l \mapsto \mathbb{Z}_{\geq 0}$ is defined as the Hamming distance between two equal-length sequences:

$$\delta(S, S') = \sum_{j=1}^l \mathbf{1}[s_j \neq s'_j] \quad (1)$$

Definition 3.1 (Valid Counterfactual Sequence; VCS[26]). Given a finite item set $\tilde{\mathcal{I}}$ and the input alphabet $\tilde{\mathcal{I}} = \mathcal{I} \cup \{0\}$ with padding, a sequence $S \in \tilde{\mathcal{I}}^l$ of length $l > 0$ and another equal-length sequence $S' \in \tilde{\mathcal{I}}^l$ constitute a valid counterfactual sequence (VCS) if they satisfy

$$S' \neq S \quad \text{and} \quad o_1^{S'} \neq o_1^S. \quad (2)$$

Among all VCSs of S , the one that minimizes $\delta(S', S)$ is called the optimal counterfactual sequence, and a VCS that additionally satisfies $\delta(S', S) \leq \varepsilon$ is called an ε -VCS [26]. The decision problem of whether an ε -VCS exists has been shown to be **NP-Complete** [26]. Intuitively, this means that exhaustively searching over all possible substitution combinations quickly becomes intractable as the sequence length and item-set size grow. It implies that no structured search can remove the worst-case complexity itself. However, as with the TSP decision problem, NP-Completeness is a worst-case statement only; actual runtime and empirical efficiency depend heavily on instance structure and the quality of the heuristic. Existing sequential-CE studies treat targeted and untargeted settings within the same problem family [26], and this work further formalizes the ranking-sensitive cases as bring-in and push-out. The above complexity result thus serves as background for understanding why the combinatorial bottleneck of discrete sequence search remains central in our ranking-constraint setting.

4 Methodology

Problem Definition. The core task in this work is ranking-constraint counterfactual generation: editing a sequence to satisfy a directional constraint on the recommendation ranking. Rather than introducing a new sequential-CE problem family, we explicitly formalize its ranking-sensitive cases as bring-in and push-out. We write the score of item a under the score vector $\mathbf{z}(S)$ from the Preliminary as $z_a(S)$, and define the rank of item a as $\text{rank}(a | S) := 1 + |\{b \in \mathcal{O}_S : z_b(S) > z_a(S)\}|$, so a smaller rank value means a higher position.

Definition 4.1 (Ranking-Constraint CE). Given a user sequence S , a task $\tau \in \{\text{bring-in, push-out}\}$, a reference item g , and a cutoff K , let counterfactual candidates be $S' \in \tilde{\mathcal{I}}^l$ of the same length. A counterfactual sequence S' is a ranking-constraint counterfactual if it satisfies

$$S' \neq S, \quad \Phi_\tau(S') = \text{true}. \quad (3)$$

The target constraints for the two tasks are as follows:

$$\Phi_\tau(S') = \begin{cases} \text{rank}(g | S') = 1, & \tau = \text{bring-in}, g \neq o_1^S, \\ \text{rank}(g | S') > K, & \tau = \text{push-out}, g = o_1^S. \end{cases} \quad (4)$$

Definition 4.2 (Validity). Let $\mathcal{U}_{\text{eval}} \subseteq \mathcal{U}$ denote the evaluation user set. Denoting the per-user validity indicator for each $u \in \mathcal{U}_{\text{eval}}$ as $\mathbf{1}[\text{valid}_u]$, the validity rate is defined as

$$\text{Validity} = \text{mean}_{\mathcal{U}_{\text{eval}}}(\mathbf{1}[\text{valid}_u]). \quad (5)$$

In the untargeted setting, $\text{valid}_u \Leftrightarrow (S' \neq S) \wedge (o_1^{S'} \neq o_1^S)$; in our ranking-constraint setting, for the task τ_u assigned to user u , $\text{valid}_u \Leftrightarrow (S' \neq S) \wedge (\Phi_{\tau_u}(S') = \text{true})$ (Definition 4.1).

In our experimental setup, only counterfactual candidates satisfying the following three constraints are admitted.

Definition 4.3 (Non-padding Constraint). Because padding tokens are not real actions, edits are permitted only at observed positions with $s_i \neq 0$:

$$s'_i \neq s_i \Rightarrow s_i \neq 0. \quad (6)$$

This excludes “formal solutions” that modify padding positions used solely to match the model's input length, and admits only genuine user-action edits as counterfactuals.

Definition 4.4 (Feasibility Constraint). Substituted items are restricted to actionable items—those not in the user's full history and not the padding token:

$$s'_i \notin \mathcal{H}_u \cup \{0\}, \quad \forall i : s'_i \neq s_i. \quad (7)$$

This prevents re-recommending already-consumed items or substituting meaningless zero tokens, retaining only counterfactuals that could plausibly be surfaced in a real service.

Remark 4.1 (Limitations of the Feasibility Constraint). The current constraint only guarantees that substituted items are “not in the history and not padding,” and does not itself ensure behavioral plausibility. We partially compensate for this through candidate retrieval that combines context scores and embedding similarity; however, richer plausibility constraints such as category coherence, popularity range, and temporal compatibility are left as future work.

Definition 4.5 (Edit-budget Constraint). To reflect the principle of minimal change in explanations, we impose an upper bound on the number of edits:

$$\delta(S', S) \leq K_{\max}. \quad (8)$$

This controls “how many actions must be changed to alter the recommendation” and suppresses unrealistic explanations obtained through excessive editing.

4.1 TRACE

TRACE addresses the diagnosed bottleneck by decomposing the search into three stages: position opening, candidate proposal, and actual-margin pruning. The subsections below detail each stage in turn.

Influence-Guided Position Selection Position selection is performed by prioritizing positions most likely to substantially shift the target constraint’s margin, thereby reducing the combinatorial breadth of beam expansion. To this end, a leave-one-out sequence S^{-i} is constructed by deactivating only the i -th item while preserving the input length, and the change in the ranking-constraint margin

$$I_i = |m(S) - m(S^{-i})| \quad (9)$$

is used as the influence score [13]. Here $m(S)$ is the task-specific margin of the current sequence: for bring-in, $m(S) = z_g(S) - z_{(1)}(S)$; for push-out, $m(S) = z_{(K)}(S) - z_{(1)}(S)$, where $z_{(K)}(S)$ denotes the K -th highest score within O_S . Note that in the bring-in case, if g is already top-1 (the success state), then $z_{(1)}(S) = z_g(S)$ and thus $m(S) = 0$.

Plausibility-Aware Candidate Retrieval. To construct a shortlist of plausible candidates, three types of candidates are gathered for each opened position: items already highly scored in the current context, items similar to the target item, and items similar to the original item at that position. TRACE normalizes these three signals to a common scale, combines them into a single retrieval score, and forwards only the top- M candidates to actual evaluation. The purpose of this stage is therefore not to evaluate the full item space, but to construct a small shortlist that reflects both target directionality and continuity with the original sequence. The first candidate source is context-score candidates: items with the highest model scores $z_a(\hat{S})$ among recommendable items when the current beam state $\hat{S}$ (the sequence being modified during search, as distinguished from the original S ; $\hat{E}$: set of already-edited positions, $\hat{\mathcal{J}}$: selected position set) is input to the recommender, where $\arg\text{Top}_{a \in \mathcal{A}}^k f(a)$ returns the top- k elements of $\mathcal{A}$ by score $f(a)$:

$$C_{\text{ctx}}(\hat{S}) = \arg\text{Top}_{a \in O_S}^{k_{\text{context}}} z_a(\hat{S}). \quad (10)$$

These candidates are items that the model is already likely to recommend naturally in the current context, and using them as substitutions helps preserve the plausibility of the resulting sequence. The second candidate source is target NN: for bring-in, items with high embedding cosine similarity to the target item g are added to bias the search in the target direction. Push-out does not require

target-dependent candidates, so this set is left empty:

$$\mathcal{N}_{\text{target}}(g) = \begin{cases} \arg\text{Top}_{a \in O_S}^{k_{\text{target-nn}}} \cos(\mathbf{e}_a, \mathbf{e}_g), & \tau = \text{bring-in}, \\ \emptyset, & \tau = \text{push-out}. \end{cases} \quad (11)$$

The third candidate source is source NN. At each position i , items with high embedding cosine similarity to the original item s_i are added to preserve semantic continuity with the original sequence:

$$\mathcal{N}_{\text{source}}(s_i) = \arg\text{Top}_{a \in O_S}^{k_{\text{source-nn}}} \cos(\mathbf{e}_a, \mathbf{e}_{s_i}). \quad (12)$$

The initial candidate set at position i for beam state $\hat{S}$ is then

$$C_i(\hat{S}) = C_{\text{ctx}}(\hat{S}) \cup \mathcal{N}_{\text{target}}(g) \cup \mathcal{N}_{\text{source}}(s_i). \quad (13)$$

To combine signals on different scales, TRACE min-max normalizes each term within $C_i(\hat{S})$ and re-ranks candidates by the following retrieval score:

$$\rho_i(a | \hat{S}) = \lambda_{\text{ctx}} \tilde{z}_a(\hat{S}) + \lambda_{\text{target}}(\tau) \widetilde{\cos}(\mathbf{e}_a, \mathbf{e}_g) + \lambda_{\text{source}} \widetilde{\cos}(\mathbf{e}_a, \mathbf{e}_{s_i}), \quad (14)$$

where $\tilde{z}_a(\hat{S})$ and $\widetilde{\cos}(\cdot, \cdot)$ are the context score and cosine similarity normalized within $C_i(\hat{S})$, and $\lambda_{\text{ctx}}=1.0$, $\lambda_{\text{target}}=1.0$, and $\lambda_{\text{source}}=0.5$ are fixed weights (for bring-in; push-out sets $\lambda_{\text{target}}(\tau)=0$ since no target item is specified). The final shortlist is

$$A_i(\hat{S}) = \arg\text{Top}_{a \in C_i(\hat{S})}^M \rho_i(a | \hat{S}). \quad (15)$$

4.1.1 Actual-Margin Beam Search. At each search depth d , the beam is updated as:

$$\mathcal{B}_d = \text{top-}B_{m(\cdot)} \left(\bigcup_{(\hat{S}, \hat{E}) \in \mathcal{B}_{d-1}} \{(\hat{S}[i \mapsto a], \hat{E} \cup \{i\}) : i \in \hat{\mathcal{J}}, a \in A_i(\hat{S})\} \right) \quad (16)$$

Here each beam element is a tuple $(\hat{S}, \hat{E})$, and $\text{top-}B_{m(\cdot)}$ selects the top- B tuples by the task-specific margin $m(\hat{S})$ of the first component. The retrieval stage $A_i(\hat{S})$ is heuristic, but beam selection always uses the actual task-specific margin; thus, even if the candidate generator is imprecise, candidates within the beam are sorted by how closely they satisfy the target constraint. When $B=1$, a single incorrect substitution can lock the search onto a failure path; maintaining $B>1$ parallel paths reduces this vulnerability.

Query cost increases whenever the recommender model is actually called. In TRACE, queries are consumed specifically at three points: (1) leave-one-out evaluations for influence computation, (2) context-score retrieval on the current beam state, and (3) actual evaluation of shortlisted child sequences. Of these, the third term typically grows fastest as the item space expands; accordingly, the primary purpose of the retrieval stage is to reduce the number of candidates forwarded to the beam and thereby control this dominant cost.

Algorithm 1 TRACE: ranking-constraint CE search

Require: Original sequence S , task τ , reference item g , cutoff K , beam size B , max edits $K_{\max}$, shortlist size M

Ensure: Ranking-constraint counterfactual $\hat{S}$, or $\emptyset$ on failure

```

1:  $\mathcal{B}_0 \leftarrow \{(S, \emptyset)\}$  ▷ beam: (sequence, edited positions)
2: for  $d = 1$  to  $K_{\max}$  do
3:    $\mathcal{P} \leftarrow \emptyset$  ▷ candidate pool at depth  $d$ 
4:   for all  $(\hat{S}, \hat{E}) \in \mathcal{B}_{d-1}$  do
5:     for all editable positions  $i \notin \hat{E}$  do
6:        $I_i \leftarrow |m(\hat{S}) - m(\hat{S}^{-i})|$ 
7:     end for
8:      $\hat{\mathcal{J}} \leftarrow \text{TOPPOSITIONS}(\{I_i\})$  ▷ influence-guided selection
9:     for all  $i \in \hat{\mathcal{J}}$  do
10:       $\hat{A}_i \leftarrow \text{SHORTLIST}(\hat{S}, i; \text{context}, \text{target-nn}, \text{source-nn})$ 
11:      for all  $a \in \hat{A}_i$  do
12:         $S' \leftarrow \hat{S}[i \mapsto a]$ 
13:        evaluate  $S'$  with the actual recommender
14:        if  $\Phi_\tau(S') = \text{true}$  then
15:          return  $S'$  ▷ constraint satisfied
16:        end if
17:         $\mathcal{P} \leftarrow \mathcal{P} \cup \{(S', \hat{E} \cup \{i\})\}$ 
18:      end for
19:    end for
20:  end for
21:  deduplicate  $\mathcal{P}$  and sort by task-specific margin  $m(\cdot)$ 
22:   $\mathcal{B}_d \leftarrow \text{top-}B_{m(\cdot)}(\mathcal{P})$  ▷ actual-margin beam pruning
23: end for
24: return  $\emptyset$ 

```

Table 1: SRS model performance (HR@10, NDCG@10).

Metric	Model	ML-100K	ML-1M	Steam	Beauty	Sports
HR@10	SASRec	.2004	.3079	.1348	.0844	.0432
	BERT4Rec	.1506	.1906	.1249	.0584	.0421
	GRU4Rec	.1654	.2445	.1196	.0602	.0360
NDCG@10	SASRec	.1016	.1776	.0726	.0519	.0243
	BERT4Rec	.0747	.0976	.0662	.0332	.0227
	GRU4Rec	.0832	.1395	.0644	.0355	.0205

4.2 Semantic-Quality Metric

Standard metrics such as validity and Hamming distance are insufficient to judge the semantic quality of generated counterfactuals. This subsection defines an auxiliary proxy metric that captures some aspects of semantic quality and naturalness, and which is used throughout the experiments (§5) as a complementary lens on counterfactual quality.

Definition 4.6 (Delta of Negative Log-Likelihood; ΔNLL). Let $\mathcal{V}$ be the set of users for which a valid counterfactual exists. For user sequence S , let $\bar{s} = [\bar{s}_1, \dots, \bar{s}_T]$ denote the non-padding subsequence of length T with padding tokens removed. The sequence negative log-likelihood (NLL) proxy used in this paper is defined as

$$\text{NLL}_{\text{AR}}(\bar{s}) = -\frac{1}{T-1} \sum_{t=2}^T \log p_{\mathcal{M}}(\bar{s}_t \mid \bar{s}_1, \dots, \bar{s}_{t-1}) \quad (17)$$

for unidirectional models such as SASRec and GRU4Rec. Bidirectional masked language models such as BERT4Rec do not provide unidirectional conditional probabilities directly; for these, we replace each position t with the [MASK] token to obtain $\bar{s}^{(t)} = [\bar{s}_1, \dots, \bar{s}_{t-1}, [\text{MASK}], \bar{s}_{t+1}, \dots, \bar{s}_T]$ and use the per-position reconstruction probabilities:

$$\text{NLL}_{\text{MLM}}(\bar{s}) = -\frac{1}{T} \sum_{t=1}^T \log p_{\mathcal{M}}(\bar{s}_t \mid \bar{s}^{(t)}). \quad (18)$$

Both definitions share the interpretation that lower values indicate sequences that are more natural under the model distribution. Denoting user u 's original and counterfactual non-padding sequences as $\bar{s}_u$ and $\bar{s}'_u$ respectively, the reported ΔNLL is computed over the valid counterfactual set $\mathcal{V}$ as

$$\Delta\text{NLL} = \frac{1}{|\mathcal{V}|} \sum_{u \in \mathcal{V}} (\text{NLL}(\bar{s}'_u) - \text{NLL}(\bar{s}_u)), \quad (19)$$

where NLL is replaced by NLL_{AR} or NLL_{MLM} depending on the model. Smaller values (and especially negative values) indicate counterfactuals with less likelihood loss—or even likelihood improvement—relative to the original. ΔNLL is a likelihood-based proxy that measures how natural the whole sequence appears under the model distribution.

5 Experiments

Explainer hyperparameters. All explainers share a query budget of $Q = 10,000$ and an evaluation set size of $|\mathcal{U}_{\text{eval}}| = 500$. The edit budget is fixed at $K_{\max} = 5$ for both tasks, and this setting is applied identically across all methods within a task. The search terminates successfully upon finding a valid ranking-constraint counterfactual, and fails upon exhausting the query budget Q or reaching the edit upper bound $K_{\max}$. The main experiments cover two ranking-constraint tasks. First, bring-in uniformly samples one target item from ranks 2–50 of the original recommendation ranking and requires raising it to top-1. Second, push-out requires moving the original top-1 item outside the top-50. All results are reported as 3-seed means over $\{42, 43, 44\}$. The TRACE configuration used in the main comparison is a beam searcher that combines influence-guided position selection based on leave-one-out margin changes with a candidate retriever that mixes context scores, cosine similarity to the target item, and cosine similarity to the current-position original item. For bring-in, we uniformly use $B=5$, $k_{\text{context}}=6$, $k_{\text{target-nn}}=3$, $k_{\text{source-nn}}=3$, and shortlist size 12 across all combinations. For push-out, we use the same beam structure but, since no specific target item exists, candidates are constructed from context scores and source similarity only.

Baselines. We compare against two external baselines. To clarify access-level differences: GECE is a pure black-box searcher that uses only score queries, while TRACE is an embedding-accessible searcher that additionally leverages learned item embedding similarity for candidate retrieval—a comparison designed to directly measure the structural gains of embedding access. First, **GECE** [26] is a GA-based sequential-recommendation counterfactual generator that maintains a population of candidate sequences and iteratively applies crossover and mutation, requiring no model information beyond score queries. Second, **Greedy** is a “best immediate action” baseline that commits to the single modification most improving

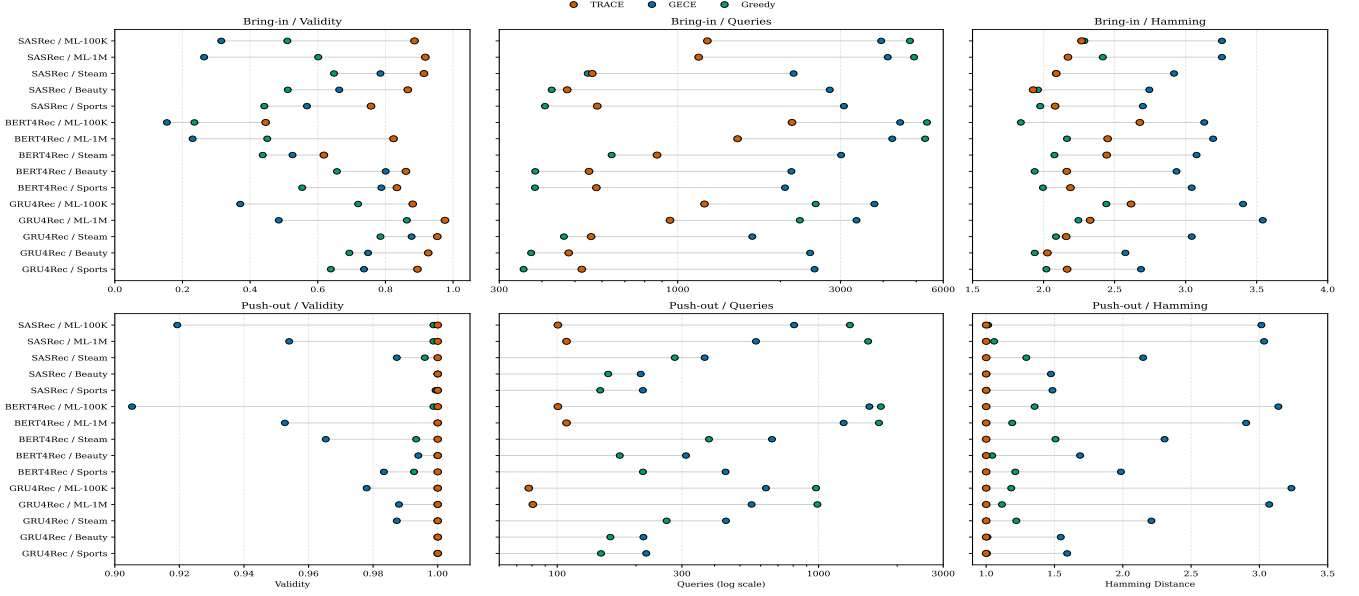


Figure 1: Per-combination comparison of TRACE, GECE, and Greedy. Each row represents one model–dataset combination. The top row shows bring-in and the bottom row shows push-out results. Within each row, the three points represent the three methods; Validity is better toward the right, and Queries and Hamming distance are better toward the left. For bring-in, TRACE is significantly better than GECE on all three metrics in all 15 combinations, and significantly higher than Greedy in validity in every combination (McNemar / paired sign test, $p < 0.05$).

Table 2: Δ NLL comparison. Each value is the conditional mean $\pm$ standard deviation over users with a valid counterfactual. Under each dataset, ‘In’ denotes bring-in and ‘Out’ denotes push-out. TRACE’s Δ NLL is significantly lower than GECE’s in all 30 model–dataset–task combinations (paired bootstrap mean-difference test over matched valid users, $p < 0.05$). Validity, Queries, and Hamming distance are visualized in Figure 1; this table provides the exact numbers for likelihood preservation.

Model	Method	ML-100K		ML-1M		Steam		Beauty		Sports	
		In	Out	In	Out	In	Out	In	Out	In	Out
SASRec	GECE	.6469 $\pm$.0290	.0692 $\pm$.0089	.5815 $\pm$.0261	.0253 $\pm$.0095	2.2039 $\pm$.0868	1.1370 $\pm$.0967	2.5760 $\pm$.0361	.4033 $\pm$.0336	1.8099 $\pm$.0518	.4650 $\pm$.0534
	TRACE	.1691$\pm$.0055	-.0249$\pm$.0003	.1285$\pm$.0059	-.0206$\pm$.0005	.3130$\pm$.0184	-.3447$\pm$.0117	.6557$\pm$.0222	-.3857$\pm$.0182	.2614$\pm$.0369	-.4503$\pm$.0126
BERT4Rec	GECE	.2570 $\pm$.0214	.2458 $\pm$.0083	.2692 $\pm$.0385	.2171 $\pm$.0163	1.9062 $\pm$.0549	2.3405 $\pm$.1098	3.8713 $\pm$.0962	1.3037 $\pm$.1615	2.4530 $\pm$.0500	2.5953 $\pm$.0449
	TRACE	.0365$\pm$.0042	-.0355$\pm$.0010	.0132$\pm$.0047	-.0329$\pm$.0005	.0397$\pm$.0236	-.3905$\pm$.0326	1.9738$\pm$.1574	-.0557$\pm$.1281	.4458$\pm$.0267	-.2923$\pm$.0148
GRU4Rec	GECE	.4750 $\pm$.0047	.1772 $\pm$.0148	.5931 $\pm$.0061	.2275 $\pm$.0073	2.1749 $\pm$.0521	1.8538 $\pm$.0351	2.7387 $\pm$.0983	.6590 $\pm$.0134	2.2052 $\pm$.0715	.6017 $\pm$.0158
	TRACE	.1494$\pm$.0094	-.0354$\pm$.0008	.1686$\pm$.0062	-.0214$\pm$.0009	.2488$\pm$.0568	-.2513$\pm$.0138	1.2661$\pm$.0413	-.1268$\pm$.0545	.6179$\pm$.0399	-.4709$\pm$.0048

the target margin at each step, representing the upper bound of single-path search.

Reproducibility details. The evaluation set $\mathcal{U}_{\text{eval}}$ is constructed by drawing 500 users uniformly at random from the sorted user set with a fixed seed. “Queries” in the reported tables is defined as the number of times a single candidate sequence is evaluated once by the model. The hardware and software environment consists of an NVIDIA GeForce RTX 5090, CUDA 12.8, and PyTorch 2.10.0+cu128. Explanation experiments are conducted on five public benchmarks: ML-100K (943 users, 1,349 items), ML-1M (6,040 users, 3,416 items [5]), Steam (334,536 users, 13,046 items), Beauty (22,363 users, 12,101 items), and Sports (35,598 users, 18,357 items [19]). Following standard protocols, we apply 5-core filtering (retaining only users and items with at least five interactions), treat all interactions as implicit feedback, and use leave-one-out splitting (last interaction for test, second-to-last for validation). We train SASRec,

Table 3: TRACE cumulative ablation on bring-in validity.

Variant	SASRec / 100K	BERT4Rec / 1M	GRU4Rec / Steam
Naive	56.7 $\pm$ 1.6	56.4 $\pm$ 2.7	63.7 $\pm$ 2.5
+Beam	60.5 $\pm$ 0.9	59.4 $\pm$ 2.1	78.4 $\pm$ 2.4
+Infl.	57.0 $\pm$ 0.2	60.5 $\pm$ 2.1	76.1 $\pm$ 1.5
Full	88.6$\pm$1.0	82.4$\pm$1.1	95.3$\pm$0.3

BERT4Rec, and GRU4Rec as backbone recommenders. Table 1 reports recommendation performance (HR@10, NDCG@10) on all five datasets. Beauty and Sports [19] are relatively sparse Amazon-domain datasets included to verify generalization across different domains and density conditions.

5.1 Main Results

This section compares TRACE with Greedy and GECE. The primary validity–queries–Hamming trade-off is presented in Figure 1, while the exact ΔNLL values for likelihood preservation are summarized in Table 2.

Bring-in. Under the same $K_{\max} = 5$ budget, TRACE is significantly better than GECE on validity, Queries, and Hamming distance in all 15 model–dataset combinations ($p < 0.05$). Compared to the stronger reference point Greedy, TRACE has significantly higher validity in all 15 combinations; Queries and Hamming distance show significant improvement in 6/15 combinations each. This suggests that, in bring-in, performance gains are driven more by structuring the search space (position selection and candidate proposal) than by simply repeating mutations.

Push-out. Across all 15 combinations, TRACE does not reduce validity relative to GECE (significance is partially attenuated by ceiling effects) and maintains the same or equivalent success rate. On efficiency metrics, TRACE records significantly fewer Queries and smaller Hamming distance than GECE in all 15 combinations. Compared to Greedy, Queries is significantly improved in 15/15 combinations and Hamming distance in 13/15. Thus, TRACE’s primary gain in push-out is not an increase in the success rate but rather an improvement in efficiency at the same success rate.

In both tasks, ΔNLL is lower for TRACE in every combination as shown in Table 2, and the differences between TRACE and GECE are significant across all 30 model–dataset–task combinations.

5.2 Ablation Study

Table 3 verifies the query bottleneck in bring-in stage by stage. Naive preserves the TRACE beam structure itself but uses only a context-top- k shortlist with beam width $B=1$. +Beam increases the beam width to maintain multiple paths. +Influence narrows the edit positions to the top few using leave-one-out position scoring, and +Full TRACE additionally applies the retrieval shortlist that incorporates target and source similarity. This cumulative comparison asks in sequence: is widening the beam sufficient? Is opening the right positions important? Is additionally proposing the right candidates necessary? For bring-in, +Full TRACE achieves the highest validity in all three representative combinations, and the differences between Full and Naive/+Beam/+Influence are all significant ($p < 0.01$). This suggests that influence-guided opening and plausibility-aware proposal are mutually complementary for improving query efficiency in bring-in.

One notable pattern is that the validity of +Influence is lower than +Beam in SASRec/ML-100K and GRU4Rec/Steam. This reveals an adverse interaction that arises when position narrowing and candidate quality are decoupled. +Influence narrows the edit positions to the top- m by leave-one-out influence, but candidates remain drawn solely from context-score top items—items that the recommender already ranks highly, meaning they compete with the target item for the rank-1 position. Placing these into the positions with the greatest influence over the target margin can inadvertently increase the rank-1 competing score $z_{(1)}(S)$ in $m(S) = z_g(S) - z_{(1)}(S)$. Simultaneously, narrowing to the top- m positions excludes from the search space solutions that require $\delta \geq 4$, which +Beam can find by freely combining over all positions (empirically: 26 of 32

Table 4: Push-out efficiency ablation on representative combinations.

		SASRec / ML-100K	BERT4Rec / ML-1M	GRU4Rec / Steam
Variant Metric				
Naive	Q	452.9±2.4	508.6±5.2	70.7±3.0
	ΔNLL	.1277±.0027	.0281±.0012	.0865±.0164
+Beam	Q	452.9±2.4	508.6±5.2	70.7±3.0
	ΔNLL	.1277±.0027	.0281±.0012	.0865±.0164
+Infl.	Q	100.6±0.3	108.5±0.7	44.4±0.6
	ΔNLL	-.0249±.0003	-.0329±.0005	-.2513±.0138
Full	Q	123.4±0.3	131.1±0.8	67.3±0.8
	ΔNLL	-.0249±.0003	-.0329±.0005	-.2513±.0138

instances where +Beam succeeds but +Influence fails require $\delta \geq 4$). Indeed, even where +Influence succeeds, the average edit distance is larger than +Beam (3.08 vs. 2.76), suggesting that influence identifies the right positions but sub-optimal candidates require more edits. Full TRACE resolves this by adding target-NN candidates. Target-NN items share high embedding similarity with the goal item and do not compete for rank 1; placing them at influential positions genuinely improves the target margin. Thus, influence (where to open) and target-NN (what to insert) are each effective only in conjunction with the other.

Table 4 reports the same cumulative ablation for push-out in terms of Q and ΔNLL . Two notable patterns emerge. First, the Q and ΔNLL of Naive and +Beam are completely identical across all three combinations. This is because push-out succeeds at depth 1 for all users (edit count $\delta=1$, validity 100%). Since the searcher evaluates all candidates in a depth- d batch before pruning the beam, if termination always occurs at depth 1, the beam width B has no effect on query count or solution quality. In other words, the benefit of beam expansion in push-out only manifests for difficult instances that proceed to depth 2 or beyond—none of which exist in push-out. Second, +Infl. has lower Q than Full but identical ΔNLL . Full adds source-similarity-based candidates (source-NN, $k_{\text{source-nn}}=3$) to +Infl., expanding the per-position shortlist from 6 context candidates to up to 9. In push-out, these additional candidates do not change the optimal solution, so ΔNLL remains unchanged, but the evaluation cost of approximately 3 additional candidates per position $\times$ position_top_m=5 positions manifests as an increase in Q (empirical difference ≈ 23 queries/user). This suggests that, unlike bring-in, source-NN in push-out consumes queries without providing additional benefit.

5.3 Qualitative Case Study

Table 5 presents one bring-in case directly comparing TRACE and GECE on the same user instance, one case demonstrating that the $K_{\max}=5$ setting genuinely leverages multiple substitutions, and one failure case that reveals the proposal bottleneck. Case A is an example from the Steam domain where target-NN retrieval enables TRACE to succeed on an instance where GECE fails, Case B illustrates the structural necessity of multiple edits in the BERT4Rec

Table 5: Qualitative cases for TRACE. “...,” denotes an omitted prefix, underline marks a substituted item, and bold marks the target item.

Case A: Direct comparison Steam domain (SASRec / Steam / User 58370)	
Original sequence	..., Thomas Was Alone (Indie), <u>The Dwarves</u> (RPG), <u>Battleborn</u> (Action), Clicker Heroes (Adventure, Casual, Free to Play, Indie, RPG, Simulation, Strategy)
Edits	The Dwarves → Fallout 4; Battleborn → DayZ
Rec. change	Grand Theft Auto V → Rust
Comparison	TRACE succeeds with $\delta=2$, Queries = 313, whereas GECE fails to find a valid ranking-constraint CE even after Queries = 5000.
Case B: Multi-edit success (BERT4Rec / Steam / User 213700)	
Original sequence	..., Saints Row: The Third (Action), <u>Magic Duels</u> (Free to Play, Strategy), <u>Killing Floor 2</u> (Action), <u>Fable Fortune</u> (Strategy, Early Access)
Edits	Magic Duels → Fallout 4; Killing Floor 2 → Deus Ex: Mankind Divided; Fable Fortune → Shadow Warrior 2
Rec. change	Grand Theft Auto V → Dishonored 2
Comparison	TRACE succeeds with $\delta=3$, Queries = 223; on the same instance, GECE requires $\delta=4$ and Queries = 900.
Case C: Bring-in failure (BERT4Rec / ML-1M / User 4527)	
Original rec. / Target	Almost Famous → Small Time Crooks
Outcome	no valid ranking-constraint CE found ($\delta=0$, Queries = 1060)
Interpretation	Many candidates are evaluated, but the target-aware retrieval shortlist itself does not contain sufficient valid candidates, causing the bottleneck to appear before beam pruning.

setting, and Case C shows an instance where valid candidates are not recovered despite consuming sufficient queries.

Case A illustrates the role of target-NN retrieval in the Steam domain. The target item Rust has embedding neighbors sharing the Survival, Open World, and Early Access tags, and TRACE’s target-NN shortlist captures DayZ (Survival, Zombies, Open World) as one of those neighbors. Because Steam tag combinations are far more complex than those in ML-1M, GECE, which must find the correct combination without directional guidance, fails even after Queries = 5000.

The $\delta=3$ solution in Case B is a structural consequence of BERT4Rec’s bidirectional architecture. Because BERT4Rec is a bidirectional encoder, the margin change induced by a single-position edit is partially offset by the remaining positions. In practice, among valid counterfactuals for BERT4Rec on ML-1M, the fraction with $\delta=1$ is only 16.4%, lower than for SASRec (23.8%), while the fraction with $\delta \geq 4$ reaches 29.5%, higher than for SASRec (23.6%). Multi-edit solutions are thus structurally more common in the BERT4Rec setting, and the $K_{\max}=5$ budget genuinely captures this pattern.

Case C reveals that the failure stems from the long-tail status of the target item rather than a simple query shortage. Small Time Crooks has only 369 ratings in ML-1M, less than half the average target item rating count for successful instances (998). Low-popularity items occupy sparse regions of the embedding space, leaving the

target-NN shortlist with too few valid candidates; as a result, the beam fails to find an entry point despite consuming 1,060 queries. Comparing success and failure instances by target item popularity rank, successful cases fall in the top 11.9% of the popularity distribution on average, while failed cases fall in the top 28.4%—a clear and consistent gap.

The common patterns revealed across these three cases can be summarized as follows. First, TRACE concentrates edits on a small number of positions through influence-guided selection: 35% of all edits land on the last position, compared with only 19% for GECE. Second, the genre/tag Jaccard similarity between original and substituted items is consistently higher for TRACE (mean 0.278) than for GECE (mean 0.177), with GECE producing substitutions that share no genre overlap in more than half of all cases. This suggests that the source-NN and context-score shortlist implicitly preserves semantic continuity as well as improving retrieval efficiency.

On the bring-in task, some users still lack a valid counterfactual even under the $K_{\max}=5$ budget (see the top row of Figure 1). Unlike push-out, which merely displaces the current top-1, bring-in requires rearranging the entire score landscape to elevate the desired item to the top. As Case C illustrates, the proposal bottleneck is especially pronounced for long-tail target items, pointing to the need for a stronger candidate generator that exploits co-occurrence priors or popularity weighting.

6 Discussion

Tradeoffs of embedding access. TRACE adopts an embedding-accessible setting, using learned item embedding similarity in addition to score queries. This is a deliberate design choice: rather than searching the full item space through undirected mutation, embedding access enables pre-filtering the candidate space into a plausibility-aware shortlist, substantially improving query efficiency. Experimental results confirm that this gain outweighs the cost of the additional access, particularly in bring-in where the score landscape is most complex. However, this setting requires that the recommender model expose item embeddings, or that they be accessible externally. In fully closed API environments where embeddings are unavailable, a pure black-box searcher—such as GECE—is more appropriate. The embedding-accessible setting that TRACE defines occupies a middle ground between pure black-box and full white-box access, and maps naturally to practical scenarios where item embeddings are served separately or the underlying model is open-source.

Limitations of the likelihood proxy. Δ NLL assumes that the recommender’s learned information and likelihood structure capture a meaningful notion of sequence naturalness, but it does not directly measure plausibility at the level of user behavior [34]. Human evaluation or validation with an independent model remains future work.

This work compares against the main baselines directly applicable to SRS: an SRS-specific CE method (GECE), a simple reference point (Greedy), and ablation variants for component analysis. CETD [6] is a learning-based approach that requires training a dedicated CE generator (a VAE), which is paradigmatically different from our training-free setting where CEs are produced at inference time without additional training of the recommender. CE methods

for non-sequential recommendation (such as PRINCE) are difficult to apply directly to sequence editing and are excluded, and adversarial-attack techniques for images or text (C&W, PGD, etc.) are based on continuous spaces and do not transfer directly to the discrete sequence setting. Second-order-information-based (Hessian) SRS CE methods have not been published to date; if such methods are proposed in the future, a direct comparison would be meaningful.

7 Conclusion

This work proposed **TRACE**, a structured search framework for ranking-constraint counterfactual explanation in sequential recommendation, built on a three-stage decomposition of position opening, candidate proposal, and actual-margin pruning. Experimental results show that TRACE achieves complete validity across all push-out combinations while using substantially fewer queries and recording lower ΔNLL than GECE. On the harder bring-in task, under the same $K_{\max} = 5$ budget, TRACE also achieves higher validity than GECE in all 15 combinations and records fewer queries in every combination. Future directions include stronger retrieval that exploits co-occurrence or behavioral priors, more rigorous user-level plausibility constraints, and validity studies based on human evaluation.

Acknowledgments

This research was supported by the Regional Innovation System & Education (RISE) program through the RISE Center, Gyeongsangnam-do, funded by the Ministry of Education (MOE) and the Gyeongsangnam-do Provincial Government, Republic of Korea (2026-RISE-16-001); the Korea Planning & Evaluation Institute of Industrial Technology (KEIT) grant funded by the Ministry of Trade, Industry and Energy (MOTIE), Republic of Korea (No. RS-2026-25607526); and the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2026-25476256).

References

- [1] Sajid Ali, Tamer Abuhmed, Shaker El-Sappagh, Khan Muhammad, Jose M. Alonso-Moral, Roberto Confalonieri, Riccardo Guidotti, Javier Del Ser, Natalia Díaz-Rodríguez, and Francisco Herrera. 2023. Explainable Artificial Intelligence (XAI): What we know and what is left to attain Trustworthy Artificial Intelligence. *Information Fusion* 99 (Nov. 2023), 101805. doi:10.1016/j.inffus.2023.101805
- [2] Azin Ghazimatin, Oana Balalau, Rishiraj Saha Roy, and Gerhard Weikum. 2020. PRINCE: Provider-side Interpretability with Counterfactual Explanations in Recommender Systems. In *Proceedings of the 13th International Conference on Web Search and Data Mining (WSDM '20)*. Association for Computing Machinery, New York, NY, USA, 196–204. doi:10.1145/3336191.3371824
- [3] Muheeb Faizan Ghori, Arman Dehpanah, Jonathan Gemmell, Hamed Qahri-Saremi, and Bamshad Mobasher. 2021. How does the User's Knowledge of the Recommender Influence their Behavior? doi:10.48550/arXiv.2109.00982 arXiv:2109.00982 [cs].
- [4] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Dino Pedreschi, Franco Turini, and Fosca Giannotti. 2018. Local Rule-Based Explanations of Black Box Decision Systems. doi:10.48550/arXiv.1805.10820 arXiv:1805.10820 [cs].
- [5] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4 (2015), 19:1–19:19. doi:10.1145/2827872
- [6] Ming He, Boyang An, Jiwen Wang, and Hao Wen. 2023. CETD: Counterfactual Explanations by Considering Temporal Dependencies in Sequential Recommendation. *Applied Sciences* 13, 20 (Oct. 2023), 11176. doi:10.3390/app132011176
- [7] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. doi:10.48550/arXiv.1511.06939 arXiv:1511.06939 [cs].
- [8] Sarthak Jain and Byron C. Wallace. 2019. Attention is not Explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 3543–3556. https://aclanthology.org/N19-1357/
- [9] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. 197–206. doi:10.1109/ICDM.2018.00035 ISSN: 2374-8486.
- [10] Amir-Hossein Karimi, Gilles Barthe, Bernhard Schölkopf, and Isabel Valera. 2021. A survey of algorithmic recourse: definitions, formulations, solutions, and prospects. doi:10.48550/arXiv.2010.04050 arXiv:2010.04050 [cs].
- [11] Amir-Hossein Karimi, Bernhard Schölkopf, and Isabel Valera. 2021. Algorithmic Recourse: from Counterfactual Explanations to Interventions. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. 353–362.
- [12] Mark T. Keane, Eoin M. Kenny, Eoin Delaney, and Barry Smyth. 2021. If Only We Had Better Counterfactual Explanations: Five Key Deficits to Rectify in the Evaluation of Counterfactual XAI Techniques. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence*. 4466–4474. doi:10.24963/ijcai.2021/609
- [13] Pang Wei Koh and Percy Liang. 2017. Understanding Black-box Predictions via Influence Functions. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, 1885–1894.
- [14] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (Aug. 2009), 30–37. doi:10.1109/MC.2009.263
- [15] Todd Kulesza, Simone Stumpf, Margaret Burnett, and Irwin Kwan. 2012. Tell me more?: the effects of mental model soundness on personalizing an intelligent agent. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, Austin Texas USA, 1–10. doi:10.1145/2207676.2207678
- [16] Todd Kulesza, Simone Stumpf, Margaret Burnett, Sherry Yang, Irwin Kwan, and Weng-Keen Wong. 2013. Too much, too little, or just right? Ways explanations impact end users' mental models. In *2013 IEEE Symposium on Visual Languages and Human Centric Computing*. IEEE, San Jose, CA, USA, 3–10. doi:10.1109/VLHCC.2013.6645235
- [17] Wenqi Li, Jui-Ching Kuo, Manyu Sheng, Pengyi Zhang, and Qunfang Wu. 2025. Beyond Explicit and Implicit: How Users Provide Feedback to Shape Personalized Recommendation Content. doi:10.48550/arXiv.2502.09869 arXiv:2502.09869 [cs].
- [18] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [19] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. 2015. Image-Based Recommendations on Styles and Substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '15)*. Association for Computing Machinery, New York, NY, USA, 43–52. doi:10.1145/2766462.2767755
- [20] Catarina Moreira, Yu-Liang Chou, Chihcheng Hsieh, Chun Ouyang, João Pereira, and Joaquim Jorge. 2025. Benchmarking Instance-Centric Counterfactual Algorithms for XAI: From White Box to Black Box. *Comput. Surveys* 57, 6 (June 2025), 1–37. doi:10.1145/3672553
- [21] Ramaravind K. Mothilal, Amit Sharma, and Chenhao Tan. 2020. Explaining machine learning classifiers through diverse counterfactual explanations. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT* '20)*. Association for Computing Machinery, New York, NY, USA, 607–617. doi:10.1145/3351095.3372850
- [22] Rafael Poyiadzi, Kacper Sokol, Raul Santos-Rodriguez, Tijl De Bie, and Peter Flach. 2020. FACE: Feasible and Actionable Counterfactual Explanations. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*. 385–392.
- [23] Massimo Quadrana, Paolo Cremonesi, and Dietmar Jannach. 2018. Sequence-Aware Recommender Systems. *ACM Comput. Surv.* 51, 4 (2018), 66:1–66:36. doi:10.1145/3190616
- [24] Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. 1994. GroupLens: an open architecture for collaborative filtering of netnews. In *Proceedings of the 1994 ACM conference on Computer supported cooperative work (CSCW '94)*. Association for Computing Machinery, New York, NY, USA, 175–186. doi:10.1145/192844.192905
- [25] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1135–1144.
- [26] Domiziano Scarcelli, Filippo Betello, Giuseppe Perelli, Fabrizio Silvestri, and Gabriele Tolomei. 2025. Demystifying Sequential Recommendations: Counterfactual Explanations via Genetic Algorithms. doi:10.48550/arXiv.2508.03606 arXiv:2508.03606 [cs].
- [27] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. Association for Computing Machinery, New York, NY, USA, 1441–1450. doi:10.1145/3357384.3357895
- [28] Juntao Tan, Shuyuan Xu, Yingqiang Ge, Yunqi Li, Xu Chen, and Yongfeng Zhang. 2021. Counterfactual Explainable Recommendation. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (CIKM*

- '21). Association for Computing Machinery, New York, NY, USA, 1784–1793. doi:10.1145/3459637.3482420
- [29] Khanh Hiep Tran, Azin Ghazimatin, and Rishiraj Saha Roy. 2021. Counterfactual Explanations for Neural Recommenders. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 1627–1631. doi:10.1145/3404835.3463005
- [30] Sandra Wachter, Brent Mittelstadt, and Chris Russell. 2018. Counterfactual Explanations without Opening the Black Box: Automated Decisions and the GDPR. doi:10.48550/arXiv.1711.00399 arXiv:1711.00399 [cs].
- [31] Tongshuang Wu, Marco Tulio Ribeiro, Jeffrey Heer, and Daniel S. Weld. 2021. Polyjuice: Generating Counterfactuals for Explaining, Evaluating, and Improving Models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, 4288–4303. doi:10.18653/v1/2021.acl-long.523
- [32] Zhichao Xu, Hansi Zeng, Qingyao Ai, and Yongfeng Zhang. 2024. CFE2: Counterfactual Editing for Search Result Explanation. In *Proceedings of the 2024 ACM SIGIR International Conference on Theory of Information Retrieval*. ACM. doi:10.1145/3664190.3672508
- [33] Yongfeng Zhang and Xu Chen. 2020. Explainable Recommendation: A Survey and New Perspectives. *Foundations and Trends® in Information Retrieval* 14, 1 (March 2020), 1–101. doi:10.1561/15000000066 arXiv:1804.11192 [cs].
- [34] Jakub Černý, Jiří Němec, Ivan Dovica, and Jakub Mareček. 2025. Plausible Counterfactual Explanations of Recommendations. doi:10.48550/arXiv.2507.07919 arXiv:2507.07919 [cs].